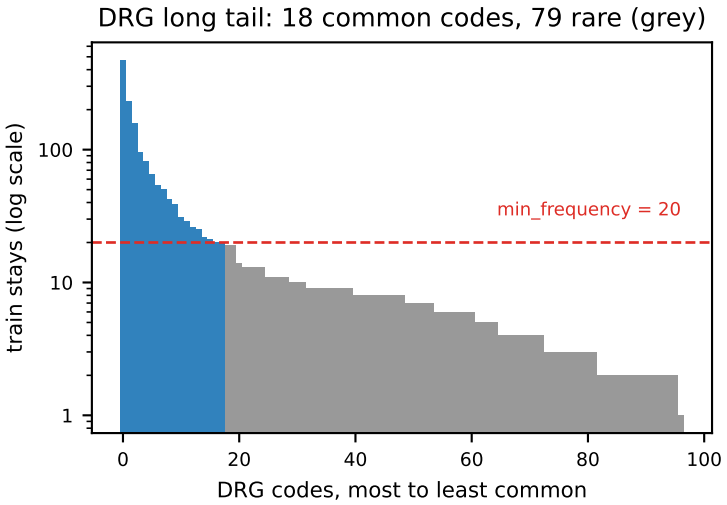
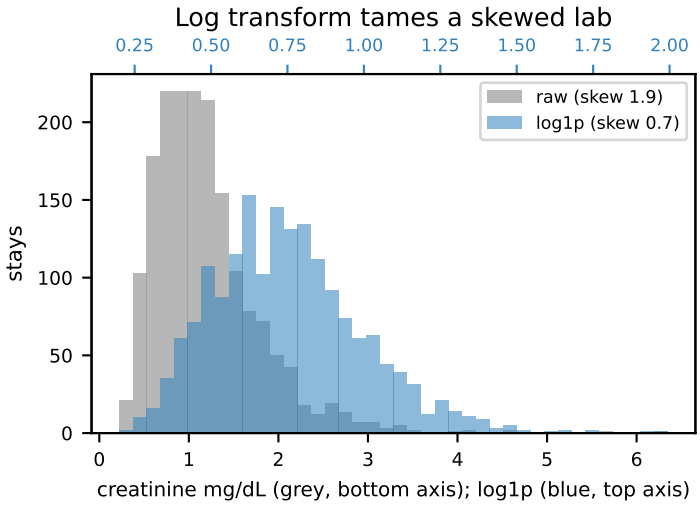
Day 29 Cheat Sheet: The Feature Engineering Toolkit

Week 5 opens: one ColumnTransformer turns messy healthcare columns into model-ready features

COLUMN TYPE	RECIPE (inside the Pipeline)	WHY
Numeric	SimpleImputer(median) + StandardScaler	fair scale for regularized models
Skewed (LOS, labs)	impute + FunctionTransformer(np.log1p) + scale	long tails stop dominating
Lab with thresholds	impute + KBinsDiscretizer(4, quantile)	catches non-linear jumps (anemia)
Missing labs	MissingIndicator / add_indicator=True	missing often means something
Low-cardinality cats	OneHotEncoder(handle_unknown='ignore')	payer, admit type, disposition
ICD-10 codes	group to chapter first: 'I50.9' -> 'I'	thousands of codes -> ~20 groups
DRG (high card.)	OneHotEncoder(min_frequency=20) or TargetEncoder	rare codes pooled, no memorizing
Dates	weekday, month, weekend, days since last visit	only look BACKWARD in time
Names	ct.get_feature_names_out()	explain every column you made



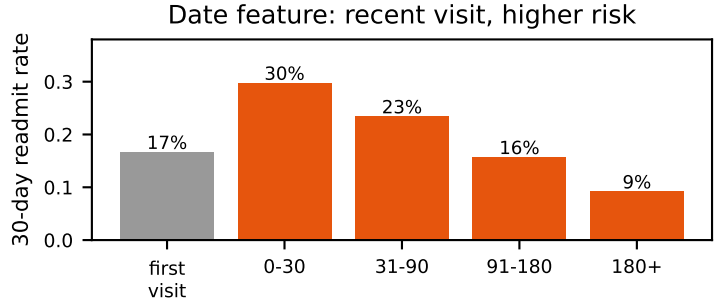


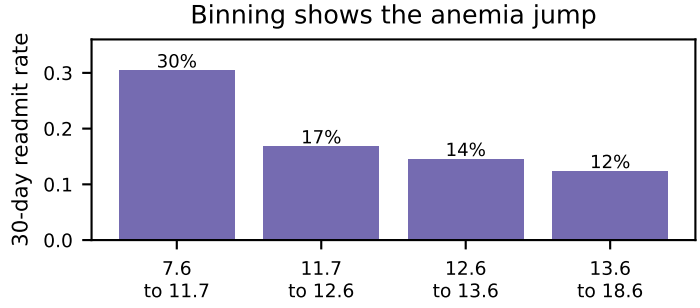
THIS DATA: 2,641 simulated stays from 1,000 patients (seed 29), 30-day readmit rate 18.4%, 100 DRG codes, split by patient:
train 1,989, test 652. 79 of 97 train DRG codes have < 20 stays (25% of stays). 3 test codes never appear in train.
5-fold GroupKFold ROC AUC, same logistic regression each time: raw numeric 0.692 | toolkit without DRG 0.714 |
+ DRG target encoded 0.719 | + DRG rare grouped 0.724 | + DRG all codes 0.731 | LEAKY DRG mean 0.752
Rule: best CV AUC minus 1 sd = 0.720; simplest that qualifies = rare grouped (55 columns vs 133). Test once: AUC 0.746 (raw numeric 0.714).
The leak: averaging readmit per DRG on all rows before CV scored 0.752; the honest encoder with the same idea scored 0.719.

RULES OF THUMB

- Anything that learns from data (medians, scales, bins, categories, target means) is fit INSIDE the pipeline, per fold.
- Row-level features that only look backward (weekday, days since last visit) can be built before the split.
- Rare levels are where models memorize. Pool them, shrink them, or cross-fit them; never raw-average on all rows.
- handle_unknown matters: next month will bring DRG codes your training data never saw.

```
ct = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), num),
    ("log", make_pipeline(SimpleImputer(strategy="median"), log1p, StandardScaler()), ["los_days", "creatinine"]),
    ("miss", MissingIndicator(), ["creatinine", "hemoglobin", "sodium"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["payer", "admit_type", "discharge", "dx_chapter"]),
    ("drg", OneHotEncoder(min_frequency=20, handle_unknown="infrequent_if_exist"), ["drg"])])
model = make_pipeline(ct, LogisticRegression(C=0.5, max_iter=3000))
```





Infographic: From messy columns to model-ready features

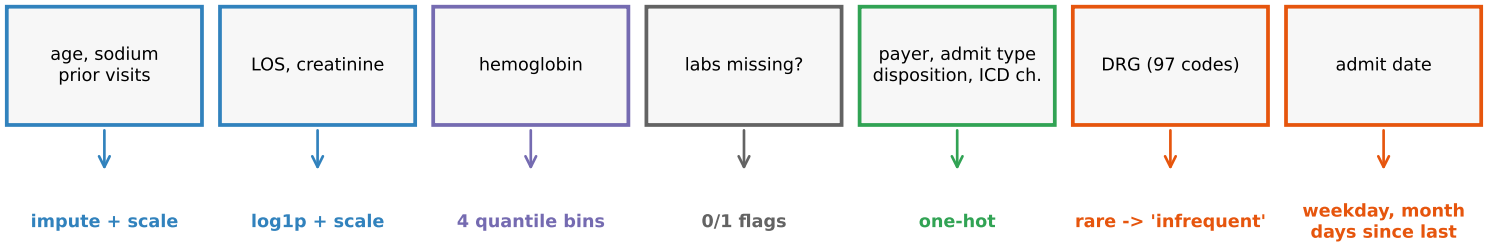
30-day readmission, simulated healthcare data, one ColumnTransformer

RAW INPUT COLUMNS
11

AFTER THE TOOLKIT
55

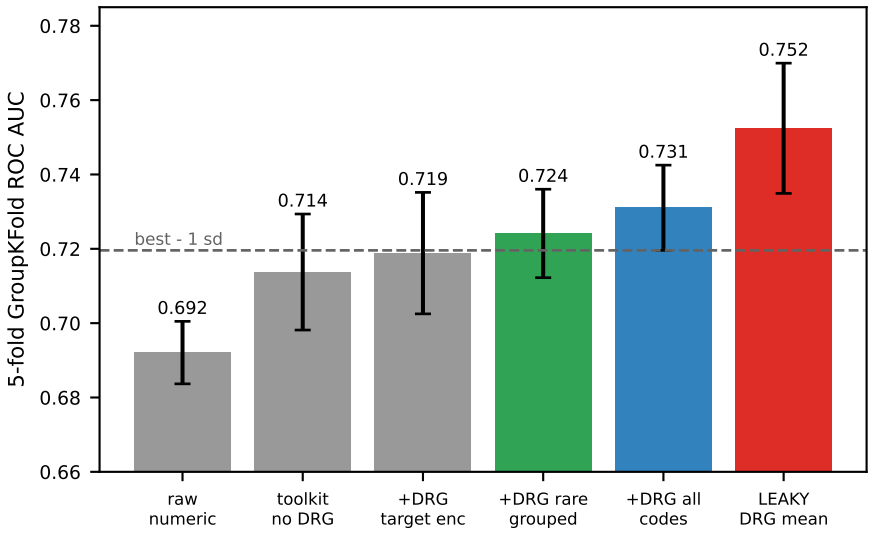
CV AUC GAIN
+0.032

TEST AUC
0.746

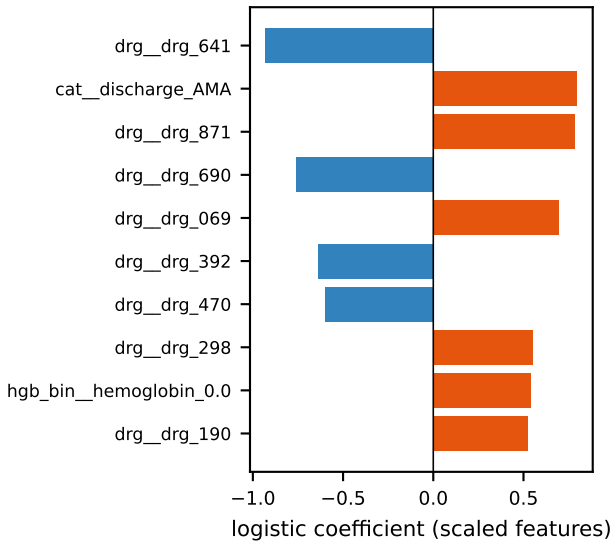


ColumnTransformer -> LogisticRegression, all inside one Pipeline, refit in every GroupKFold fold

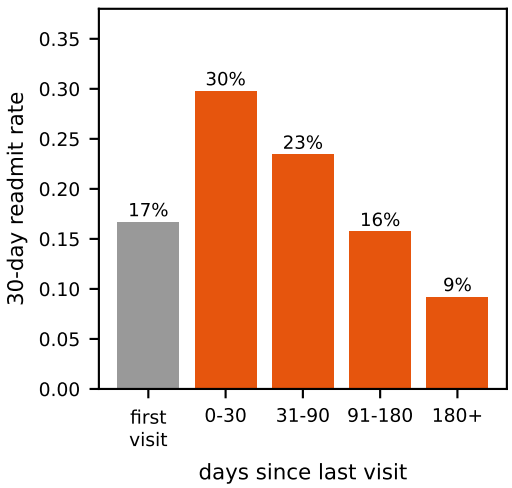
Same model, different feature recipes (green = champion, red = leak)



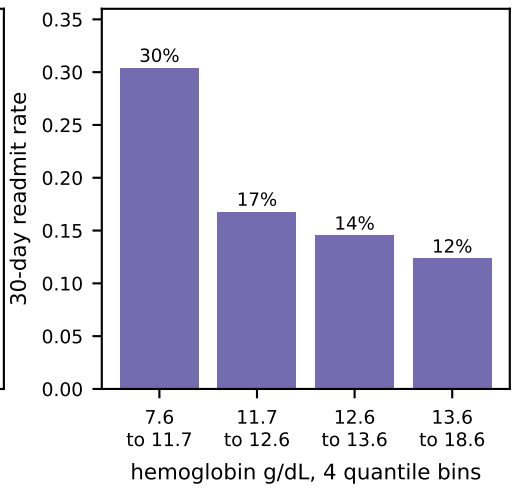
Top 10 weights, by name



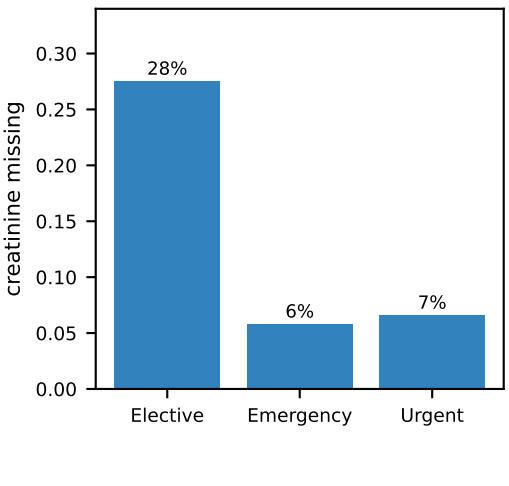
Date feature: recent visit, higher risk



Binning shows the anemia jump



Missing is not random: flag it



How to read it

1. The toolkit alone (no DRG) lifts AUC from 0.692 to 0.714: logs, bins, flags and date features add real signal.
2. DRG adds the most. All codes (0.731) and rare grouped (0.724) are within one sd; grouped uses 55 columns, not 133.
3. The red bar is a lie: DRG readmit means computed on all rows before CV let each fold peek at its own answers.
4. Recent visits (0-30 days: 30%) and low hemoglobin (30%) carry risk a single linear term would blur.

Hands-On Worksheet: ds-genai-90-days / day29

60-90 minutes | notebook: day29/readmit_features.ipynb + day29/README.md + feature_list.json

[ ] mkdir day29, conda activate ds90, open day29/readmit_features.ipynb in VS Code

[ ] Simulate 2,641 stays from 1,000 patients, seed 29; save readmit_week5.csv (snippet 1)

[ ] Split by patient with GroupShuffleSplit; set up GroupKFold(5)

[ ] Add backward-looking features: weekday, month, days since last visit, ICD chapter (snippet 2)

[ ] Count DRG levels on TRAIN; how many have fewer than 20 stays?

[ ] Build the ColumnTransformer: scale, log1p, bins, flags, one-hot (snippet 3)

[ ] Print get_feature_names_out() and explain 5 of the columns out loud

[ ] Race 5 honest recipes with the same model and folds (snippet 4)

[ ] Run the LEAKY recipe and write one sentence on why it scores higher

[ ] Apply the rule: best CV AUC, simplest within 1 sd (snippet 5)

[ ] Refit the champion on TRAIN, score the test set ONCE

[ ] Plot top coefficients by name; save the model + feature_list.json

[ ] README: recipe table, leak story, chosen encoding, limits (simulated)

[ ] git add day29 && git commit && git push (snippet 6)

[ ] Stretch: build the same features in Snowflake SQL (snippet 7)

```
ct = prep("rare_onehot").fit(train, ytr)        # fit on TRAIN only
names = ct.get_feature_names_out()             # 'drg__drg_infrequent_sklearn', 'miss__...'
TargetEncoder(target_type="binary", smooth="auto", cv=KFold(5, shuffle=True))   # cross-fitted
```

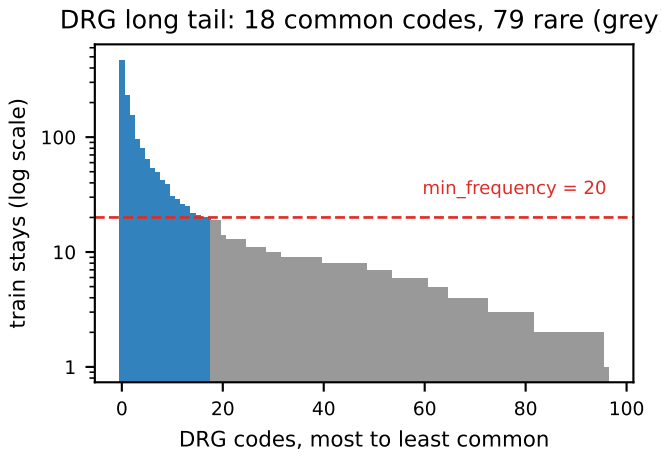
DRG codes in train: _____ with < 20 stays: _____

Columns after toolkit: _____ raw CV AUC: _____

Champion recipe: _____ CV AUC: _____

Test AUC: _____ Leaky CV AUC: _____

Notes: _____



Day 29 Self-Check Quiz

1. Why put imputers, scalers and encoders inside the Pipeline instead of running them on the full table first?

_____

2. Which features are safe to build before the train/test split, and why?

_____

3. What does OneHotEncoder(min_frequency=20, handle_unknown='infrequent_if_exist') do with rare and new DRG codes?

_____

4. Why did the 'DRG mean before CV' recipe score higher in CV than the honest TargetEncoder?

_____

5. Creatinine has a long right tail. What transform helps a linear model, and why?

_____

6. 25% of elective stays have no creatinine. What do you add, and why not just fill the median?

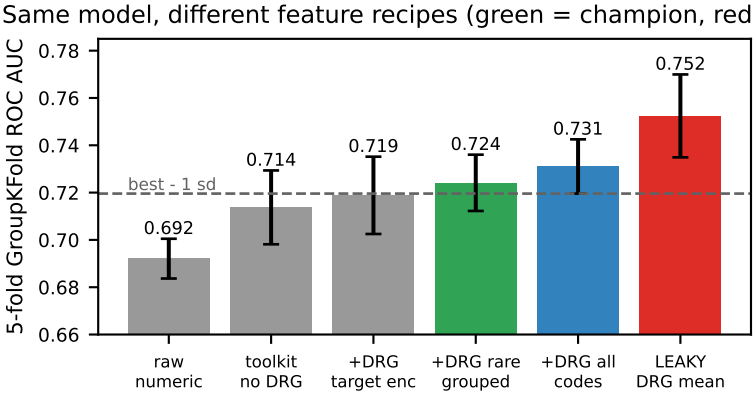
_____

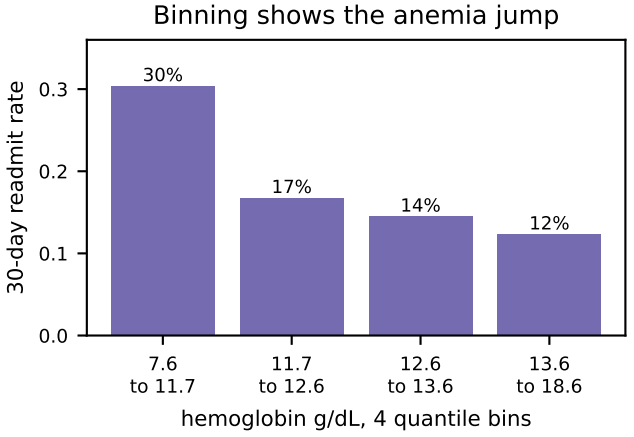
7. 'All codes' beat 'rare grouped' by 0.007 AUC. Which do you ship, and why?

_____

8. What does get_feature_names_out() give you, and when do you need it?

_____





Answers:
1 each CV fold must learn medians, scales, bins and categories from its own training part only; otherwise validation rows leak in
2 row-level, backward-looking ones: weekday, month, days since the SAME patient's previous visit, ICD chapter; they use no other rows' labels
3 codes with < 20 training stays share one 'infrequent' column; unseen codes at predict time go there too, so nothing crashes
4 each validation row's own label was inside its DRG average (0.752 vs 0.719); rare codes memorize the answer
5 log1p: it pulls in the tail (skew 1.9 -> 0.7) so a few extreme values do not dominate the slope
6 a missing-indicator flag: missing is not random here (elective 28% vs emergency 6%), and the flag lets the model learn that
7 rare grouped: the gap is inside 1 sd (0.011), it uses 55 columns instead of 133, and it is easier to explain
8 the name of every output column (e.g. 'log__creatinine'); you need it to read coefficients, importances, and to document features